

Om at skrive kode fra formler

Ikke så sjældent, skal vi lave et program der baseres på en formel, og skive noget kode som implementerer formelen, på de data der er tilgængelige for programmet.

Formlen er oftest en matematisk formel.

Vi skal kigge på at bryde formler ned i dele, og sætte dem sammen igen i et program.

Vi skal bruge modulet `math`

```
In [1]: import math
```

Symboler til kode

4 hurtige...

Matematik	Python
π	<code>math.pi</code>
$\sqrt{x}$	<code>math.sqrt(x)</code>
x^2	<code>x**2</code>
x^n	<code>x**n</code>

π (Pi)

π har jo noget at gøre med cirklers omkreds osv.

En cirkel med radius 10 (mm), har omkreds

$$10 \cdot \pi = 31,4$$

I Python bruger vi konstanten `pi`, fra modulet `math`.

```
In [2]: math.pi
```

```
Out[2]: 3.141592653589793
```

```
In [3]: 10 * math.pi
```

```
Out[3]: 31.41592653589793
```

Formlen for arealet af en cirkel er

$$A_{cirkel} = \pi \cdot r^2$$

f.eks hvis radius $r = 5$

$$\pi \cdot 5^2 = 78,54$$

```
In [4]: math.pi * 5**2
```

```
Out[4]: 78.53981633974483
```

I øvrigt kan en kugles areal ud regnes med formlen

$$A_{kugle} = 4 \cdot \pi \cdot r^2$$

i python bliver det:

```
In [5]: # hvis radius = 2  
r = 2  
4 * math.pi * r ** 2
```

```
Out[5]: 50.26548245743669
```

Læs mere om kugler på <https://da.wikipedia.org/wiki/Kugle>

Det væsentlige er at *oversætte* fra formel til kode!

$$A_{kugle} = 4 \cdot \pi \cdot r^2$$

I kugle-eksemplet brugte jeg en variabel `r` fordi den var i formlen. Det kunne også være at den var konstant i det aktuelle program.

Men, mere sandsynlig, vil jeg gerne udregne arealer af flere kugler med forskellig *radius*, så jeg laver en funktion:

```
In [6]: def A_kugle(r):  
        return 4 * math.pi * r ** 2  
  
print(A_kugle(5))
```

```
314.1592653589793
```

Note

Resultatet her, arealet af en kugle med radius = 5, er π fordi $r = 5$ og $5^2 = 25$.

Hvis vi indsætter 5 for r , og bytter om på rækkefølgen mellem gangetegnene i formlen (må man gerne), får vi :

$$A_{kugle} = 4 \cdot \pi \cdot r^2$$

$$A_{kugle} = 4 \cdot \pi \cdot 5^2$$

$$A_{kugle} = 4 \cdot \pi \cdot 25$$

$$A_{kugle} = \pi \cdot 4 \cdot 25$$

$$A_{kugle} = \pi \cdot 100$$

Et lidt mere indviklet eksempel

En kugles rumfang (volumen).

Egentlig er det lige meget med hvilke fomler det er!

Bare vi kigger på at oversætte matematik til kode (i python)

$$V_{kugle} = \frac{4}{3} \cdot \pi \cdot r^3$$

Nogen gange dropper man gange-tegnene i formlen (fordi de er indforståede):

$$V_{kugle} = \frac{4}{3} \pi r^3$$

$$V_{kugle} = \frac{4}{3} \cdot \pi \cdot r^3$$

```
In [7]: def V_kugle(r):  
        return 4/3 * math.pi * r**3  
  
V_kugle(5)
```

```
Out[7]: 523.5987755982989
```

Et endnu værre eksempel

Breregn *radius* når vi kender *rumfanget*

$$r_{kugle} = \left(V_{kugle} \cdot \frac{3}{4 \cdot \pi} \right)^{\frac{1}{3}}$$

Faktisk er det *3. rod af ...* altså $r_{kugle} = \sqrt[3]{\left(V_{kugle} \cdot \frac{3}{4 \cdot \pi}\right)}$ men den findes ikke indbygget i pythons `math`, så vi laver et trick med at opløfte i $1/3$ potens, som er det samme :-)

$$r_{kugle} = \left(V_{kugle} \cdot \frac{3}{4 \cdot \pi}\right)^{\frac{1}{3}}$$

```
In [8]: def r_kugle(v):  
        return v * 3 / 4 * math.pi ** 1 / 3  
  
        r_kugle(1000)
```

Out[8]: 785.3981633974482

785 er en lidt stor radius for et rumfang på 1000!

Vi skal sørge for at bruge parenteser!

Nå man læser en formel, er der lige som nogen implicitte parentser, som vi skal sørge for at lave selv, når man omsætter formeln til programkode.

Især hvis:

- der står flere led under en brøkstreg
- potenser af noget der består af flere led

$$r_{kugle} = \left(V_{kugle} \cdot \frac{3}{(4 \cdot \pi)}\right)^{\left(\frac{1}{3}\right)}$$

```
In [9]: def r_kugle(v):  
        return (v * 3 / (4 * math.pi)) ** (1 / 3)  
  
        r_kugle(1000)
```

Out[9]: 6.203504908994

Er det ikke rimeligt realistisk at der kan være en liter mælk (1000 ml), i en kugle med en *radius* på 6.2 cm? Husk radius er fra kant til centrum, så diameteren, tværs over kuglen (kant til kant), er ca 12.4 cm...

Hellere for mange parentereser, end for få

$$r_{kugle} = \left(V_{kugle} \cdot \left(\frac{3}{(4 \cdot \pi)} \right) \right)^{\left(\frac{1}{3} \right)}$$

```
In [10]: def r_kugle(v):  
          return (v * (3 / (4 * math.pi))) ** (1 / 3)  
  
          r_kugle(1000)
```

Out[10]: 6.203504908994

 Tabel over precedens regler

Se <https://qissba.com/operators-precedence-in-python/>

Matematik med gentagelser

Man kan godt beskrive visse typer af gentagelser på strukturerede data, som lister, med matematiske formler. Den slags formler kommer i til at møde i data-engineering og data-analysis, derfor denne lille introduktion :-)

Vi skal kigge på symbolerne Σ (og måske Π)

Vi kender fra python at vi kan have en liste, f.eks `x = [...]`.

I python ville vi referere til at elementerne i listen med indexerings "firkant-parenteserne",

f.eks. værdien på plads 2 (det 3. for vi starte på 0) med `x[2]`

Vi siger at `x` har `n` elementer, og indekserer med variabelen `i`.

```
In [11]: x = [5,8,9,2]  
          n = len(x)  
          for i in range(n):  
              print(f"{i}. element i x er: {x[i]}")
```

```
0. element i x er: 5  
1. element i x er: 8  
2. element i x er: 9  
3. element i x er: 2
```

Lidt mere besværligt end vi plejer at gøre med python og lister, men for lige at være sikker på at have notationen med...

Sum Σ

Nu vil jeg gerne beregne summen af tallene i listen `x`, så jeg *kunne* bare skrive `sum(x)`, men jeg vil jo gerne vise noget mere med det...

```
In [12]: resultat = 0
         for i in range(n):
             resultat += x[i]

         print(x)
         print(resultat)
```

```
[5, 8, 9, 2]
24
```

Summen af tallene i `x`, defineres matematisk med formlen:

$$\sum_{i=0}^{n-1} x_i$$

Nogen gange i kort form: $\sum_{i=0}^{n-1} x_i$

Nogen gange skrives det mere ud som:

$$\sum_{i=0}^{n-1} x_i = x_0 + x_1 + \dots + x_3$$

Med listen `x` ([5, 8, 9, 2]) vil det sige:

$$\sum_{i=0}^{n-1} x_i = x_0 + x_1 + x_2 + x_3 = 5 + 8 + 9 + 2 = 24$$

Med andre ord, $\sum_{i=0}^{n-1} x_i$ betyder at man tager alle elementer i `x` (x_i) og lægger sammen!

Hvor `i` starter på 0 og går op til `n-1`.

Da `n` er længden af `x`, så da indekset starter på 0, er det sidste indeks én før længden, så at sige.

Wow wow wow, wait-a-minute

$$\sum_{i=0}^{n-1} x_i = x_0 + x_1 + \dots + x_3 = 5 + 8 + 9 + 2 = 24$$

er det samme som denne python kode:

```
In [13]: resultat = 0
         for i in range(n):
             resultat += x[i]
```

```
print(x)
print(resultat)
```

```
[5, 8, 9, 2]
24
```

Gennemsnit af x

I matematik notation kan man sommetider skrive gennemsnittet af x med symbolet $\bar{x}$,
men hvordan defineres og beregnes det?

Lidt mere populært kan sige gennemsnittet af en liste er:

summen af alle tallene i listen,
divideret med antallet af tal i listen

ligner noget vi kender...

$$\bar{x} = \frac{\sum_{i=0}^{n-1} x_i}{n}$$

Hvor x stadig er listen og n er længden af listen.

```
In [14]: def gns(x):
          n = len(x)
          s = 0
          for i in range(n):
              s += x[i]
          return s/n

          print(gns(x))
```

```
6.0
```

Naturligvis kunne vi bare have skrevet således, **men** *det havde ikke understreget pointen*:

Se også f.eks.: <https://favtutor.com/blogs/average-list-python>

```
In [15]: def gns(x):
          s = 0
          for tal in x:
              s += tal
          return s/len(x)

          gns(x)
```

```
Out[15]: 6.0
```

eller

```
In [16]: def gns(x):  
         return sum(x)/len(x)  
         gns(x)
```

Out[16]: 6.0

eller

```
In [17]: from statistics import mean  
         mean(x)
```

Out[17]: 6

Standardafvigelse

På engelsk, *standard deviation*

$$SD = \sqrt{\frac{\sum_{i=0}^{n-1} (x_i - \bar{x})^2}{n - 1}}$$

Standarsavigelsen er et mål for hvormeget tallene i en liste (eller dataset), afviger fra gennemsnittet. Hvis der er mange freaky undtagelser, er der en større standard afvigelse

Hvis du vil regne og forstå, så prøv [Khan Academy](#)

Lad os pille formlen ned i mindre dele så vi lettere kan løse opgaven at lave den til et program.

$$SD = \sqrt{\frac{\sum_{i=0}^{n-1} (x_i - \bar{x})^2}{n - 1}}$$

for at starte med at kigge på Σ -delen, alene, så vi opfinder en variabel s , som placeres inde i formlen, på Σ -delens plads:

$$SD = \sqrt{\frac{s}{n - 1}}$$

hvor

$$s = \sum_{i=0}^{n-1} (x_i - \bar{x})^2$$

Vi starter altså med at kigge på $s = \sum_{i=0}^{n-1} (x_i - \bar{x})^2$ Og husker at gennemsnittet

$$\text{er: } \bar{x} = \frac{\sum_{i=1}^n x_i}{n}$$

Når listen `x` ([5, 8, 9, 2]) vil det sige: $\bar{x} = \frac{5+8+9+2}{4} = \frac{24}{4} = 6$

Så kan vi bruge $\bar{x}$ i formlen

$$s = \sum_{i=0}^{n-1} (x_i - \bar{x})^2$$

$$s = \sum_{i=0}^{n-1} (x_i - \bar{x})^2$$

Σ -sybolet betyder at vi skal udregne summen af alle $(x_i - \bar{x})^2$ dvs at når $n = 4$:

$$s = (x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + (x_3 - \bar{x})^2 + (x_4 - \bar{x})^2$$

Vi indsætter her de rigtige tal for x_i med hvert af tallene i listen `x`, sådan her

når `x=[5, 8, 9, 2]`

og gennemsnittet $\bar{x} = 6$

$$s = (5 - 6)^2 + (8 - 6)^2 + (9 - 6)^2 + (2 - 6)^2$$

og så regner jeg parenterne ud,

$$s = (-1)^2 + (2)^2 + (3)^2 + (-4)^2$$

opløfter i 2'en

$$s = 1 + 4 + 9 + 16$$

og lægger sammen

$$s = 30$$

For at lave kode, kigger vi trinene med $s = \sum_{i=0}^{n-1} (x_i - \bar{x})^2$

hvor programmet skal gøre dette:

$$s = (x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + (x_3 - \bar{x})^2 + (x_4 - \bar{x})^2$$

```
In [18]: x = [5, 8, 9, 2]
n = len(x)
gns = sum(x)/len(x)
print(gns)
```

```
s = 0
for i in range(n):
    s += (x[i] - gns)**2

print(s)
```

6.0
30.0

Nu kan vi kigge på den samlede formel, og bruge at vi har udregnet `s` :

$$SD = \sqrt{\frac{s}{n-1}}$$

Vi skal bare være lidt forsigtige med parenteserne, Lad os prøve indefra

```
In [19]: n = len(x)
```

```
In [20]: s / (n - 1)
```

```
Out[20]: 10.0
```

```
In [21]: math.sqrt( s / (n-1))
```

```
Out[21]: 3.1622776601683795
```

Og så lige pænt ind i en funktion:

```
In [22]: def SD(x):
    n = len(x)
    gns = sum(x)/n

    s = 0
    for i in range(n):
        s += (x[i] - gns)**2

    return math.sqrt( s / (n - 1))

print( SD([5, 8, 9, 2]) )
```

```
3.1622776601683795
```

Koden til at beregne `s` , kan også skrives på en mere pythonistisk måde:

```
In [23]: def SD(x):
    n = len(x)
    gns = sum(x)/n

    s = 0
    for xi in x:
        s += (xi - gns)**2

    return math.sqrt( s / (n - 1))
```

```
print( SD([5, 8, 9, 2]) )
```

3.1622776601683795

Eller med list-comprehension, som minder mere om formelen:

```
In [24]: s = sum( [(xi - gns)**2 for xi in x] )  
s
```

Out[24]: 30.0

```
In [25]: def SD(x):  
    gns = sum(x)/len(x)  
    s = sum( [(xi - gns)**2 for xi in x] )  
  
    return math.sqrt( s / (len(x) - 1) )
```

Men, hvorfor ... ?

(ej skal vi nu også forstå matematikken?)

Et eksempel, med hunde (fra <https://www.mathsisfun.com/data/standard-deviation.html>)

Vi har målt 5 hundes højder til 600mm, 470mm, 170mm, 430mm and 300mm.

Altså har vi et dataset $x = [600, 470, 170, 430, 300]$

Genemsnittet er $\bar{x} = \frac{600+470+170+430+300}{5} = \frac{1970}{5} = 394$

Vi kan nu beregne hverenkelt hunds afvigelse fra gennemsnittet:

$[(600 - 394), (470 - 394), (170 - 394), (430 - 394), (300 - 394)] = [206, 76, -$

Og så regner vi ud at Standard afvigelsen er ... 147

$$SD = \sqrt{\frac{\sum_{i=0}^{n-1} (x_i - \bar{x})^2}{n - 1}} = \sqrt{\frac{206^2 + 76^2 + (-224)^2 + 36^2 + (-94)^2}{5 - 1}} = 147$$

Med Standardafvigelsen kan vi se hvormeget hundenes højder afviger fra gennemsnittet, og ...

Standardafvigelsen (SD) er således defineret, at hvis man tager alle tallene i listen, og udvælger dem, som er to standardafvigelser under gennemsnittet, op til dem, som er to standard afvigelser over gennemsnittet,

$$\bar{x} - 2 \cdot SD < x_i < \bar{x} + 2 \cdot SD$$

så har man 95% af tallene, hvis tallene er normal-fordelt!